Ghid student

Cum se desfășoară cursul

Proiectele sunt A (facturi), B (angajați noi) și C (aprovizionare). În faza 2 robotul lucrează doar prin API, iar în faza 3 automatizarea folosește un model de limbaj (LLM), cu propria cheie de model.

Grupa	Faza 1: cod	Faza 2: UiPath Studio	Faza 3: agent UiPath
Grupa 1	A	B	C
Grupa 2	B	C	A
Grupa 3	C	A	B

Grupa stabilește doar ce proiect faceți în fiecare fază. Fiecare student lucrează singur: are cheia, repository-ul, predarea și nota lui și nu preia cod de la colegi. Fazele sunt independente între ele. Profesorul vă comunică grupa și termenele.

Puteți folosi orice instrument AI, în orice fază, și nu se depunctează. Se notează rezultatul și dacă înțelegeți soluția și o puteți modifica.

Ce primiți de la profesor

- Enunțul proiectului vostru.
- Hub-ul, la `https://apb.majordom.tech`: simulează sistemele companiei și vă notează.
- Cheia personală, secretă: în variabila de mediu `BPA_KEY`, niciodată în git sau în fișierele predate. Hub-ul înregistrează fiecare rulare și fiecare cerere făcută cu cheia voastră, iar profesorul le vede.
- Dosarul „Instrumente”: runner-ul `bpa_runner.py` (Python 3.10+) și API-ul proiectelor în OpenAPI; `CITESTE.txt` spune ce fișier folosiți.
- Pentru fazele 2 și 3: „05 UiPath - fazele 2 și 3”.

Cum lucrați cu hub-ul

Rularea și BASE_URL

O rulare este o copie nouă a datelor pe hub, cu propriul `BASE_URL`; o pornire este o execuție a programului pe o rulare. Soluția primește doar acest URL și nu trimite nicio cheie. Căile din enunț sunt relative la `BASE_URL`; `{BASE_URL}/docs` și OpenAPI dau forma exactă a apelurilor.

- Data de lucru vine din `GET /context`, câmpul `business_date`. Nu folosiți ceasul calculatorului.
- Parametrii regulilor (praguri, cote, matrice) vin din `GET /config`, nu din constante în cod: notarea folosește și alte valori.

Suită	Cine o rulează	Ce conține
`public`	voi	scenariile publice, seed 1, data de lucru 2026-11-16
`sandbox`	voi	aceleași scenarii; `--seed` alege valorile și identificatorii; data de lucru și configurația rămân cele publice
`private`	profesorul	scenarii ascunse, aceleași reguli, altă dată de lucru

Suită	Cine o rulează	Ce conține
`change-request`	profesorul; voi, după ce o deschide (`--change-request` cu id-ul ei)	scenariile ascunse ale cererii de modificare

O rulare expiră după 24 de ore și acceptă cel mult 20 de cereri pe secundă și 5000 în total. Aveți cel mult 3 rulări deschise și 60 de rulări noi pe oră.

Runner-ul

Runner-ul creează rularea, pornește soluția, închide rularea și afișează raportul.

```
export BPA_HUB=https://apb.majordom.tech
export BPA_KEY=cheia-personala
python bpa_runner.py --project A --suite public -- python solutie.py {base_url}
python bpa_runner.py --project A --suite sandbox --seed 42 \
  -- python solutie.py {base_url}
python bpa_runner.py --project A --suite public \
  --kill-after-writes 5 --invocations 2 -- python solutie.py {base_url}
python bpa_runner.py --project A --suite public --manual
```

- Runner-ul înlocuiește `{base_url}` cu URL-ul rulării, disponibil și în `BPA_BASE_URL`. În PowerShell scrieți `"{base_url}"` între ghilimele și `$env:BPA_KEY = "..."`.
- `--scenarios A-P03,A-P07` rulează doar scenariile date.
- `--kill-after-writes 5 --invocations 2` testează repornirea: hub-ul suspendă rularea după a 5-a scriere înregistrată, runner-ul oprește programul și îl pornește încă de două ori (trei porniri în total).
- `--manual` (UiPath) afișează `BASE_URL` și așteaptă Enter.

Raportul și tracing

Raportul dă totalul, scorul pe categorii și verificările eșuate, cu eticheta elementului din scenariu (de exemplu `inv2.single_decision: expected 1, got 2`). Categoriile: `rules` (o regulă și limitele ei), `combined` (reguli împreună), `variations` (formate de intrare), `robustness` (erori, idempotență, reporniri), `change_request`.

Rulările `public` și `sandbox` arată fiecare verificare, cu valoarea așteptată; celelalte, doar totalul și categoriile.

Urma (trace) arată fiecare cerere: statusul, eroarea injectată, cheia, corpul. Acolo căutați întâi (`run_id` îl afișează runner-ul):

```
curl -H "Authorization: Bearer $BPA_KEY" "$BPA_HUB/v1/runs/<run_id>/trace"
```

Reguli comune tuturor proiectelor

Date

- Sumele sunt șiruri zecimale (`"1234.50"`): tip zecimal exact, nu float; rotunjire half-up doar unde cere enunțul.
- Ignorați câmpurile necunoscute din răspunsuri. În cereri trimiteți doar câmpurile definite; un câmp în plus primește 422 `VALIDATION_ERROR`.
- Un POST reușit primește 201 (`POST /report`: 200). O eroare are corpul `{"error": {"code": "...", "message": "...", "details": {}}}`; codurile din documente sunt în `error.code`.

Cheia de idempotență

Fiecare POST are o cheie în antetul `Idempotency-Key` (fără ea: 400 `IDEMPOTENCY_KEY_REQUIRED`). O cerere repetată cu aceeași cheie și același corp primește răspunsul memorat (replay) și nu înregistrează nimic nou; cu alt corp primește 422 `IDEMPOTENCY_KEY_REUSED`.

Cheia o dă enunțul (de exemplu `{document_id}:decision`); nu o generați aleator și nu folosiți ora. Listele din corp au mereu aceeași ordine. Două chei pentru aceeași acțiune dau un duplicat.

În afara raportului, 422 `IDEMPOTENCY_KEY_REUSED` înseamnă că o pornire anterioară a înregistrat deja scrierea: citiți-o din hub, nu schimbați cheia.

Politica de reîncercare

Hub-ul injectează erori. Politica, pe apel:

- 500, 502, 503, 504, eroare de rețea sau timeout: cel mult 3 încercări în total. Înainte de reîncercare așteptați `Retry-After` secunde (1 dacă lipsește).
- 429 `RATE_LIMITED`: așteptați `Retry-After` (sau 1 secundă) și retrimiteți, fără să consumați o încercare. Renunțați la apel după 10 astfel de așteptări.
- 429 `RUN_REQUEST_LIMIT`: rularea și-a epuizat cererile; ieșiți cu cod diferit de 0.
- Alte 4xx nu se reîncearcă: în afara cazurilor din enunț (de exemplu 404 „nu există”), apelul a eșuat.
- Timeout de client 10 secunde; aceeași cheie și același corp la fiecare încercare; fiecare reîncercare apare în log.

Exemplu: 503, 429, 503, 201 este un apel reușit la a treia încercare. Un 503 poate sosi după ce scrierea s-a făcut; retrimisă cu aceeași cheie, cererea nu creează duplicat.

Dacă apelul eșuează și după politică: unde enunțul descrie eșecul definitiv, urmați-l și continuați. Altfel scrieți în log, nu mai trimiteți nicio scriere și ieșiți cu cod diferit de 0, fără să inventați un rezultat de business.

503 `RUN_PAUSED` apare doar la testele de repornire: tratați-l ca pe orice 503, iar runner-ul oprește programul între timp.

Reporniri

Soluția poate fi oprită oricând și pornită din nou pe același `BASE_URL`. Trebuie să ajungă la aceeași stare finală ca o rulare neîntreruptă, fără să înregistreze ceva de două ori. Nu țineți nimic important doar în memorie sau în fișiere locale; enunțul spune ce citiți înapoi din hub. În A și B, `source` din istoric arată cine a înregistrat: `EARLIER_RUN` (o rulare anterioară) sau `THIS_RUN` (o pornire a voastră în această rulare).

Regulile fazei 1

- Orice limbaj de programare.
- Soluția primește `BASE_URL` ca prim argument sau din `BPA_BASE_URL`, procesează tot și iese cu codul 0.
- Log pe stdout sau stderr: o linie pe element de lucru, cu rezultatul și motivele, și o linie pe reîncercare.

La notare, tagul predat rulează într-un container Docker Linux. README-ul are secțiunea „Cum se rulează la notare”, cu exact trei linii, într-un bloc de cod:

```
image: python:3.12-slim
setup: pip install -r requirements.txt
command: python solutie.py {base_url}
```

- `image`: o imagine Docker publică cu limbajul vostru (implicit `python:3.12-slim`).
- `setup`: o comandă shell care instalează și compilează tot. Rulează o dată, din rădăcina repository-ului, cu internet, cel mult 30 de minute. Fără dependențe, scrieți `setup:` fără valoare.
- `command`: comanda soluției, cu `{base_url}`, rulată din rădăcina repository-ului.

La rulare containerul ajunge doar la hub, fără internet: programul nu poate apela alte servicii, inclusiv modele AI. Containerul are 2 GB memorie și 2 CPU, iar o pornire durează cel mult 15 minute.

Înainte de termen, clonați repository-ul într-un dosar gol și urmați exact cele trei linii. Dacă soluția nu poate fi rulată așa, o rulați voi într-o sesiune supravegheată.

Ce predați

- Codul (faza 1) sau proiectul UiPath (fazele 2 și 3), în locația menționată din Moodle, pentru fiecare fază.
- README: dependențe, ipoteze, cum se rulează.
- Logul soluției la o rulare publică, în repository.
- Faza 1: o notă de arhitectură (1-2 pagini) cu componentele, unde stă fiecare regulă și cum funcționează reîncercările, cheile și repornirile.
- Tagul `phaseN-submission` (N este faza), împins înainte de termen - salvat într-un git local în folderul soluției.

Cum se notează

Partea automată a notei vine doar din ce a înregistrat hub-ul. Rulările voastre nu se notează.

Componentă	Cum se măsoară
Teste publice	suita publică, rulată o dată la notare, pe tagul predat
Teste private	rulate normal, cu reporniri și cu alte valori de configurare
Cererea de modificare	scenariile ei ascunse, pe tagul `phaseN-cr`
Discuția individuală	circa 10 minute, cu fiecare student

Cererea de modificare se publică după termenul de predare. O implementați într-o fereastră scurtă și împingeți tagul `phaseN-cr`. La notare, rezultatul se compară cu versiunea predată și se caută regresii în suita privată.

La discuție răspundeți singuri, despre propria soluție.